TOPIC 00

Introduction to Machine Learning

What machine learning is, the data vocabulary, model types, and how to choose the variable you want to predict

WHAT YOU WILL LEARN

- List the **prerequisites** for machine learning and where they are taught in this repo
- Explain **what ML is**, what a **model** is, and how it differs from ordinary programming
- Describe the **types of ML** (supervised, unsupervised, reinforcement) and the **tasks** (regression, classification, clustering, dimensionality reduction), and why each is needed
- Define **independent** and **dependent** variables and name their many synonyms
- **Analyse a dataset and choose the dependent variable (Y)** with a checklist
- Explain **how that choice changes model accuracy**, with real numbers
- Read the standard **workflow**, the train/test split and the basic evaluation metrics

NOTE **Where this connects**

This is the first Machine Learning topic. Earlier folders gave you the ingredients: Python and Pandas (01_Python), cleaning and exploring data (05_EDA, whose class notes already introduce Y and X and the three problem types), and statistics (08_Statistics). The regression notes in 08_Statistics/03 taught the *statistics* of one model, linear regression. Here we step back and see where it sits among all ML models.

1. Prerequisites

You do not need to be an expert in any of these, but you should be comfortable with the **basics** of each.

Area	What you need	Why ML needs it	Where in this repo
Python	Variables, lists, dictionaries, loops, functions	All ML code is written in Python	01_Python
NumPy and Pandas	Arrays, DataFrames, selecting columns, filtering, groupby	Data is loaded, cleaned and split with them	01_Python/14_Numpy, 16_Pandas
Matplotlib / Seaborn	Histograms, scatter plots, box plots	You must *look* at data before modelling	01_Python/15, 17
EDA	Missing values, outliers, encoding, univariate and bivariate analysis	Models are only as good as the cleaned data	05_EDA
Descriptive statistics	Mean, median, variance, standard deviation, correlation	Describing data and judging relationships	08_Statistics/01
Inferential statistics	Sampling, distributions, hypothesis tests, p-values	Knowing whether a pattern is real or chance	08_Statistics/02
Regression	Line of best fit, R^2, adjusted R^2, regression table	Linear regression is the first ML model	08_Statistics/03

Area	What you need	Why ML needs it	Where in this repo
Light maths	y = mx + c, summation sign, the idea of a slope, vectors and tables of numbers	Models are formulas; training is finding good numbers for them	School algebra is enough to start

> **NOTE** **What you do NOT need yet**
>
> Advanced calculus, linear algebra proofs, or deep-learning theory. They help later, but every algorithm in this course can be learned first by *using* it and understanding the idea, and deepening the maths afterwards.

> **KEY IDEA** **Tools you will install**
>
> Python 3, Jupyter Notebook (or VS Code), and the libraries `numpy`, `pandas`, `matplotlib`, `seaborn`, `scikit-learn` (the standard ML library) and `statsmodels` (the statistics view of regression).

2. What is machine learning?

2.1 Programming versus learning

In **traditional programming** a person writes the rules and the computer applies them to data to produce answers. In **machine learning** the person supplies *data and the known answers*, and the computer **finds the rules itself**. Those discovered rules are the **model**.

	Traditional programming	Machine learning
You provide	Data + hand-written rules	Data + known answers (examples)
The computer produces	Answers	Rules (the model), then answers for new data
Example	If marks ≥ 40 then Pass	From thousands of past emails marked spam or not, learn what spam looks like
Works best when	Rules are simple and known	Rules are too many or too hard to write by hand

> **DEFINITION** **Machine learning**
>
> The study of algorithms that **improve at a task through experience (data)** instead of being told every rule. Tom Mitchell's classic definition (1997): a program learns from experience **E** with respect to a task **T** and performance measure **P** if its performance on T, measured by P, improves with E. Example: T = predict house prices, E = past sales, P = average prediction error.

2.2 AI, ML and deep learning

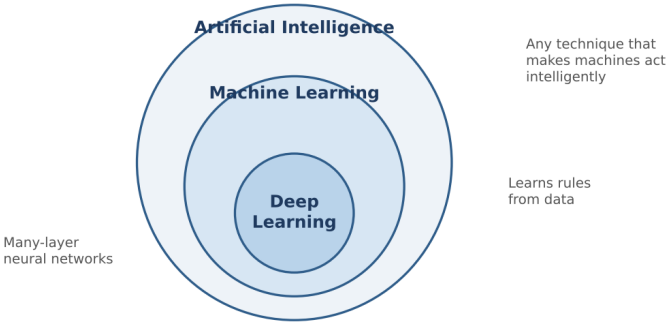


Figure 1. Machine learning is one way of building artificial intelligence; deep learning is one family of ML that uses many-layer neural networks.

2.3 When ML is, and is not, the right tool

Use ML when	Do not use ML when
The rules are hard to write (recognising faces, spam, prices)	A few clear rules solve it (tax slab, pass/fail at 40)
You have enough relevant examples with known outcomes	You have almost no data (scikit-learn's own guide says to get more data if you have under about 50 samples)
Patterns change and the system should adapt	The cost of any error is unacceptable and you cannot check it
A rough but fast prediction beats a perfect slow one	You only need a summary of the past: descriptive statistics may be enough

3. The language of data

ML borrows words from statistics, databases and computer science, so the *same thing* has many names. Learn them once and every tutorial becomes readable.

	Independent variables X features, predictors, inputs			**Dependent variable Y** label, target, output
	Age	**Experience**	**City**	**Salary**
row 1	34	2	Mumbai	5,000
row 2	45	3	Bangalore	10,000
row 3	50	4	Bangalore	15,000
new row	29	?	Pune	?

Rows 1-3 are the training data: X and Y are both known. The last row is new: the model sees X and predicts the missing Y.

(each row = one example / observation / record; each column = one variable / attribute / feature)

Figure 2. A dataset is a table. The columns we use to predict are X; the column we want to predict is Y. The class notes' example, with a new row whose Salary is unknown.

Concept	Also called	Meaning
Row	Example, observation, record, instance, sample, data point	One case: one person, house, transaction
Column	Variable, attribute, feature, field	One property measured for every row
Independent variable (X)	Feature, predictor, input, explanatory variable, regressor	What the model **uses** to make the prediction
Dependent variable (Y)	Target, label, response, output, outcome	What the model **predicts**
Dataset	Sample, data frame, sheet	All the rows together

4. What is a model?

DEFINITION **Model**

A **mathematical function that turns inputs (X) into a predicted output ($\hat{Y}$)**. Google's ML course puts it as 'the complex collection of numbers that define the mathematical relationship from specific input feature patterns to specific output label values'. A model is what you get *after* training; it is not the data and not the algorithm.

$$\hat{Y} = f(X) \qquad Y = f(X) + \varepsilon$$

The hat means **predicted**. The extra term ε is the **error**: the part of Y that X cannot explain (chance, unmeasured causes, measurement noise). No real model predicts perfectly.

4.1 A model you already know

The line from your regression notes **is** a model. Fitting the SAT and GPA data of 84 students gave:

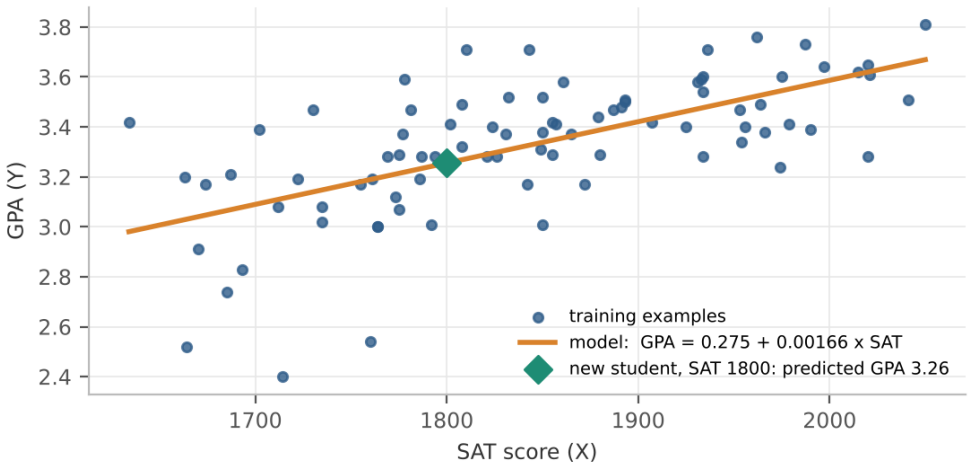


Figure 3. The model is the line GPA = 0.275 + 0.00166 × SAT. The two numbers 0.275 and 0.00166 are its **parameters**, found by training. For a new student with SAT 1800 it predicts GPA 3.26.

Word	Meaning	In the SAT example
Algorithm	The recipe for learning from data	Ordinary least squares: choose the line with the smallest squared errors
Training	Running the algorithm on data so the model finds good parameters	Finding $b_0 = 0.275$ and $b_1 = 0.00166$
Model	The fitted function	GPA = 0.275 + 0.00166 × SAT
Parameters	Numbers the model **learns** from data	b_0 and b_1
Hyperparameters	Settings **you choose before** training	e.g. the polynomial degree, or K in KNN
Inference / prediction	Using the trained model on new rows	SAT 1800 gives GPA 3.26
Loss (error) function	A score of how wrong the predictions are; training tries to make it small	Sum of squared errors (SSE)

COMMON MISTAKE **Be careful with extrapolation**

The training SAT scores run from 1634 to 2050. The model should only be trusted inside that range. Predicting for SAT = 1200 would be a guess far outside anything it has seen.

5. Types of machine learning

ML is divided in two ways: **by what the data looks like** (learning signal) and **by the task** the model performs. Choosing wrongly here is the most expensive early mistake, so learn the map.

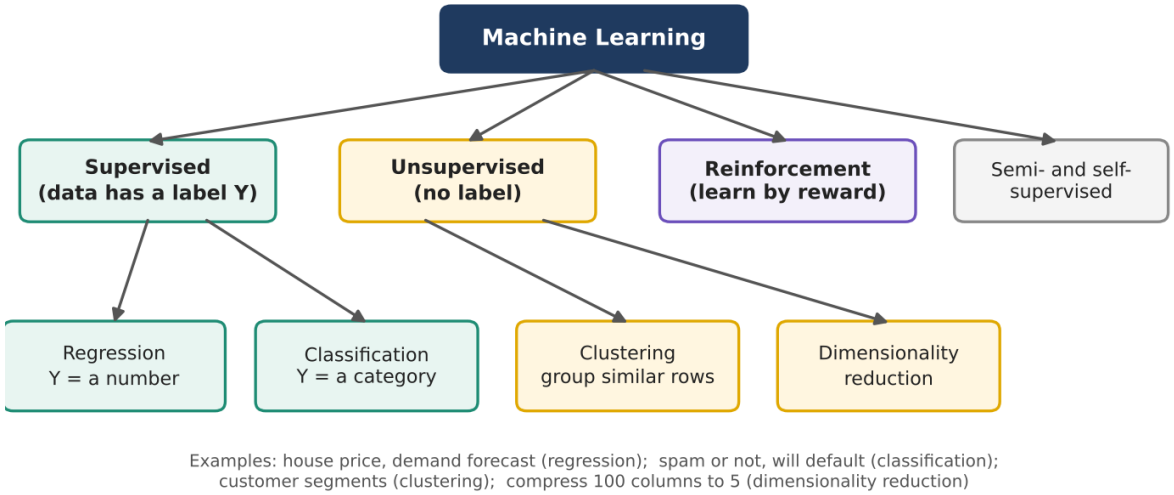


Figure 4. The main branches of machine learning. Supervised learning has a label Y; unsupervised learning does not.

5.1 By learning signal

Type	Data has a Y?	Idea	Why it is needed	Examples
Supervised	Yes, every row	Learn the link between X and a known Y, then predict Y for new rows	Prediction and decisions: most business ML	House price, loan default, spam, disease diagnosis
Unsupervised	No	Find structure in X alone	Y does not exist or labelling is too costly; discover groups, simplify data	Customer segments, compress 100 columns to 5, anomaly detection
Reinforcement	No; a reward after actions	An agent learns by trial and reward	Sequences of decisions where there is no single right answer per step	Game playing, robot control, ad bidding
Semi- and self-supervised	Few labels, or labels created from the data itself	Mix a little labelled with a lot of unlabelled data	Labels are expensive; most data is unlabelled	Modern language and image models

5.2 By task

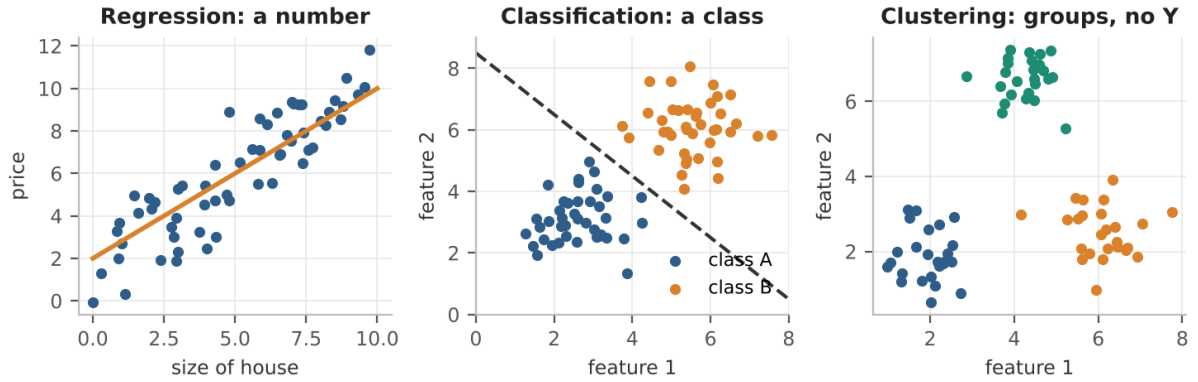


Figure 5. Regression predicts a number, classification predicts a class, clustering finds groups when there is no Y.

Task	Y looks like	Output	Why it is needed	Typical algorithms	Example
Regression	A continuous number	A number	Forecasting amounts, planning, pricing	Linear, Ridge, Lasso, KNN, trees, forests, boosting, SVR, neural networks	Predict a flat's price, tomorrow's sales
Classification	Categories: two (binary) or more (multi-class)	A class, and often a p robability	Yes/no and which-kind decisions	Logistic regression, KNN, trees, Naive Bayes, forests, boosting, neural networks	Will the customer leave: yes or no?
Clustering	No Y	A group number for each row	Discover natural segments when no labels exist	K-means, hierarchical clustering	Group customers by behaviour

Task	Y looks like	Output	Why it is needed	Typical algorithms	Example
Dimensionality reduction	No Y	Fewer, new columns	Too many columns slow models and hide patterns; helps plotting	PCA	Compress 100 sensor columns to 5

KEY IDEA **The one-line rule**

Look at Y. **Number with many values: regression. Category: classification. No Y at all: clustering (or dimensionality reduction).** The class notes use exactly this rule, and the scikit-learn 'choosing the right estimator' map begins with the same four branches.

6. Independent and dependent variables

6.1 Definitions

DEFINITION **Dependent variable (Y)**

The **outcome** we want to predict or explain. Its value *depends on* the others. In supervised learning with one target there is **exactly one** Y.

DEFINITION **Independent variables (X)**

The **inputs** used to predict Y. There can be **many** (X_1, X_2, X_3, ...). They are 'independent' only in the sense that we treat them as given inputs.

$$Y = f(X_1, X_2, \ldots, X_k) + \varepsilon$$

6.2 Facts that trip up beginners

- **Roles belong to the question, not the column.** Age is an X when predicting salary but could be the Y when predicting age from a purchase history.
- **Exactly one Y, many X's** in the standard setting. With several outputs we build several models or use special multi-output models.
- **An X may be numeric or categorical.** Categories such as City must be **encoded** into numbers before most models can use them (the EDA 'variable transformation' step).
- **Identifiers are not features.** A row number or customer ID (like the *Number* column in the practice toy dataset) carries no real information. Leave it out.
- **X and Y are about prediction, not cause.** An X predicting Y does not mean X causes Y (see the correlation versus causation notes in 08_Statistics/03).
- **The class-note equation** $Y = X_1 + X_2 + X_3$ is a simplification. A real linear model is $Y = b_0 + b_1X_1 + b_2X_2 + b_3X_3 + \varepsilon$, where the weights b are learned from data.

6.3 Relevant and irrelevant X

Every X must be checked for **relevance** to Y. Irrelevant columns add noise, slow training and can make the model fit chance patterns (overfitting). Some X columns also **duplicate each other** (for example height in cm and height in inches); this redundancy is **multicollinearity** and makes the weights unstable. The adjusted R^2 of Topic 07 in the regression notes is the statistician's tool for punishing useless variables.

7. How to analyse the data and choose the dependent variable

Choosing Y is not a technical step; it is a **problem-definition** step. A perfectly built model for the wrong Y is useless. Work through the stages below in order.

Stage 1: Start from the decision, not from the data

Ask: **what decision will someone make with this prediction?** The Y is the quantity that decision depends on.

Business question	Decision	Candidate Y	Type of problem
Which customers will leave?	Whom to offer a discount	Churned in next 90 days (yes/no)	Classification
How much will we sell next month?	How much stock to order	Units sold next month	Regression
Which loans are risky?	Approve or reject	Defaulted within 2 years (yes/no)	Classification
What kinds of customers do we have?	Design separate campaigns	None: look for groups	Clustering

Stage 2: List the candidate targets and test each one

Write every column that could answer the question, then put each through this checklist. A candidate that fails a **must** item is rejected.

#	Test	Must?	Why it matters
1	Does it answer the business question and drive an action?	Must	Otherwise the model is accurate about the wrong thing
2	Is it actually **recorded** as a column, reliably, for past rows?	Must	Supervised learning needs known answers to learn from
3	Will its value be **unknown at prediction time**, with the X values available then?	Must	If you already know Y, you do not need a prediction
4	Does it **vary**? (not constant; not 99.9% one value)	Must	No variation means nothing to learn
5	Enough rows, including enough of each class?	Must	Rare outcomes need many rows to find any pattern
6	Is it measured **consistently** (same definition over time)?	Should	Changing definitions create noisy labels
7	Is **no X derived from Y** or recorded after Y happened?	Must	Prevents data leakage (Section 8)

Stage 3: Inspect each candidate

1. **List columns and types** (Pandas: `df.info()`). This tells you whether each candidate is numeric or categorical, which fixes the problem type.
2. **Univariate analysis of Y.** Missing share; for a number, a histogram and skewness; for a category, the share of each class (`value_counts(normalize=True)`).
3. **Bivariate analysis against the X columns.** Correlation or scatter plots for numbers; group means, box plots or cross-tables for categories. Does anything move with Y?

4. **Compute a baseline.** Predict the average (regression) or the most common class (classification). Any real model must beat this.
5. **Fit a quick simple model** and compare with the baseline. If it cannot beat the baseline, the problem is the data or the choice of Y, not the algorithm.

Stage 4: Worked example on the repo's practice dataset

The file `toy_dataset.csv` in 08_Statistics/03/99_Practice_Notebook has 150,000 rows and the columns Number, City, Gender, Age, Income, Illness. Which column could be Y?

Candidate Y	Type	Evidence	Verdict
Number	ID	Just a row number	Reject: not a real quantity
Gender, City	Category	No business question here; the columns are descriptive inputs	Better kept as X
Income	Regression	Correlation with Age is −0.0013; a regression of Income on Age gets test R^2 = −0.0000	Passes tests 1 to 5 on paper, but the X columns carry no signal
Illness	Classification	Yes in 8.09% of rows, so a model that always says No is 91.91% accurate	Passes, but heavily imbalanced

Averages tell the same story: mean Income is about 91,251 for people without illness and 91,278 with it, and the illness rate is about 8% in every city and for both genders. In this dataset (a toy dataset that behaves like randomly generated data) the columns are essentially unrelated, so **no model can predict one from the others**. The skill being taught is to **find that out early**, with a baseline, instead of tuning algorithms for weeks.

> **KEY IDEA** **Takeaway**
> A good Y is (1) tied to a real decision, (2) known for past rows, (3) unknown when you must predict, (4) varied, (5) predictable from the X columns you can actually get. Test number 5 can only be checked with data: baseline first, then a simple model.

8. How the choice of Y affects model accuracy

The same dataset can give a superb model for one Y and a hopeless one for another. All numbers below are computed from the two datasets used in this course.

Effect	What happens	Evidence
1. Predictability ceiling	Accuracy can never exceed what the X columns contain about Y	SAT to GPA: hold-out $R^2 = 0.40$. Age to Income: $R^2 = -0.00$. Same algorithm, different Y.
2. Imbalanced classes (accuracy paradox)	A model can look accurate while predicting nothing useful	Illness: always answering No gives 91.9% accuracy and 0% recall. A model fitted to the data does exactly that (the snippet in Section 11). Re-weighting to catch cases raises recall to 45% but accuracy falls to 54% and precision is only 8%.
3. Turning a number into classes loses information	Converting a numeric Y to categories changes the question and the score	GPA split at the median (3.38): classification accuracy 75% (5-fold) versus a 51% baseline. Two students at 3.37 and 3.39 now sit in different classes and the model can no longer say how high a GPA is.
4. Data leakage	An X that is secretly a copy of Y gives fantasy accuracy that disappears in real use	Adding a column equal to GPA plus a little noise lifts hold-out R^2 from 0.40 to 0.97. At prediction time that column does not exist.
5. Swapping X and Y changes the question	Same data, different model, different use	Predicting SAT from GPA gives SAT = 1029 + 245 × GPA. With one X the R^2 is the same (0.406), but the model answers another question; only one of them matches your decision.
6. Noisy or shifting labels	If the recorded Y is wrong or its definition changes, the model learns the noise	Always check how Y was recorded and by whom.

COMMON MISTAKE **Never compare scores across different Y's**

R^2, error values, accuracy and recall depend on the target and its scale. 'Model A has 0.40 and model B has 0.92' means nothing unless both predict the **same Y on the same rows**.

Data leakage in detail

Data leakage means the model sees information at training time that it will **not have in real use**, so it looks excellent in testing and fails in production. Reputable guides name three common forms:

1. **Target leakage:** an X is derived from Y or recorded after it (for example 'cancellation date' when predicting churn).
2. **Train-test contamination:** preparing the data (scaling, filling missing values) on the **whole dataset before splitting**, so test information leaks into training. Split first, fit the preparation on the training part only.
3. **Time leakage:** using future values to predict the past or present. For time-ordered data, split by date, not randomly.

9. Choosing the independent variables (X)

- **Relevance:** keep columns that plausibly relate to Y; drop IDs and columns with no variation.
- **Availability:** every X must be available **at prediction time**.
- **Redundancy:** drop one of two nearly identical columns (check correlation between X columns).

- **Preparation:** encode categories, treat missing values and outliers, and scale where the algorithm needs it (EDA steps 4 to 7).
- **Create better columns:** combining columns (for example experience per year of age) can help more than a fancier algorithm (variable creation, EDA step 7).
- **Keep it simple:** compare models with adjusted R^2, cross-validation, or hold-out error, not with training score alone.

10. The machine learning workflow

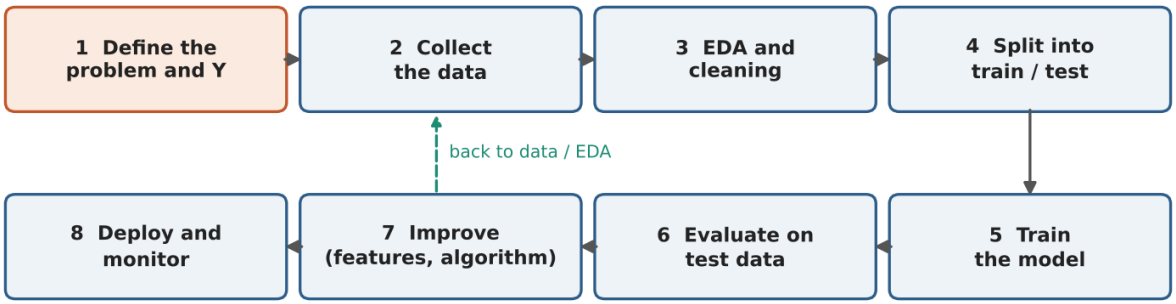


Figure 6. The eight-step cycle. The work loops back: weak results usually send you back to the data, not to a fancier algorithm.

Step	What you do	Repo link
1 Define problem and Y	Section 7 of these notes	This topic
2 Collect data	Files, databases, APIs	06_SQL
3 EDA and cleaning	Variable identification, univariate and bivariate analysis, outliers, missing values, transformation, variable creation	05_EDA
4 Split	Hold back test data	Below
5 Train	Fit the model to the training part	Next topics
6 Evaluate	Score on the test part with suitable metrics	Below
7 Improve	Better features, tuning, another algorithm	Next topics
8 Deploy and monitor	Use it for real and retrain as data changes	Later topics

10.1 Train and test split

A model must be judged on rows it **has never seen**. So we split the data, typically **70 to 80% for training and 20 to 30% for testing** (`train_test_split`). Judging a model on its own training data rewards memorising. For small data, **cross-validation** repeats the split several times.

10.2 Underfitting and overfitting

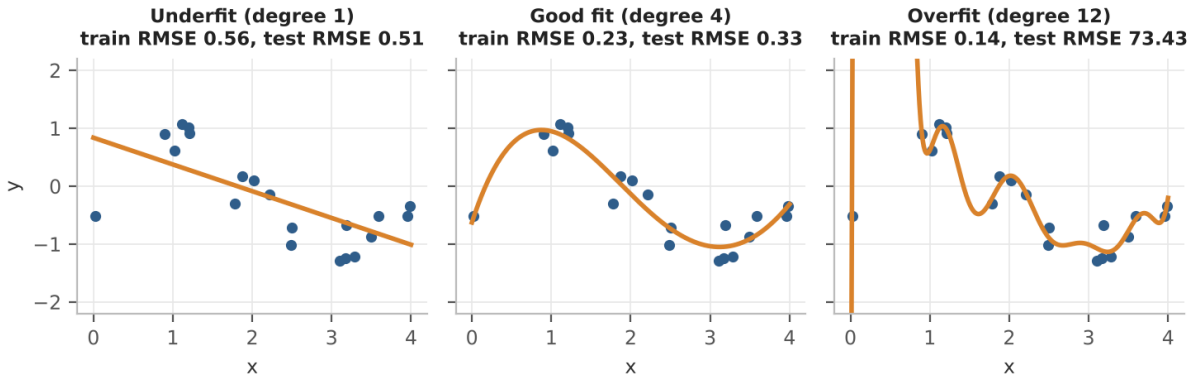


Figure 7. The same noisy curve fitted three ways. Training points: 20; test points: 200. Degree 12 reaches the training points (RMSE 0.14) but its test error is huge (73.4).

	Underfitting	Good fit	Overfitting
Model is	Too simple	About right	Too flexible, memorises noise
Training error	High	Low	Very low
Test error	High	Low	High
Fix	More features, more flexible model	Keep	More data, simpler model, regularisation (Lasso, Ridge), cross-validation

10.3 Measuring a model

Which metric you use depends on the **type of Y** (the class notes list them under 'where statistics fits in').

Regression metrics

Metric	Formula in words	Reading
MAE	Average of |actual − predicted|	Typical error in the units of Y
MSE	Average of (actual − predicted)²	Punishes large errors heavily; units are squared
RMSE	Square root of MSE	Typical error in the units of Y, large errors weighted more
R^2	1 − SSE/SST	Share of the variation in Y explained (Topic 04)
Adjusted R^2	R^2 penalised for the number of X's	Fair comparison between models (Topic 07)

Tiny example: actual = 3, 5, 2, 7; predicted = 2.5, 5, 4, 8. Errors = 0.5, 0, −2, −1. MAE = 0.875, MSE = 1.3125, RMSE = 1.146, R^2 = 0.644.

Classification metrics

A **confusion matrix** counts correct and wrong predictions per class. Example with 200 patients: 50 truly ill (positive) and 150 healthy.

	Predicted ill	Predicted healthy
Actually ill	TP = 40	FN = 20
Actually healthy	FP = 10	TN = 130

Metric	Formula	Value	Reading
Accuracy	(TP + TN) / all	0.850	Share of all predictions that are right; misleading when classes are imbalanced
Precision	TP / (TP + FP)	0.800	Of those flagged ill, how many really are
Recall	TP / (TP + FN)	0.667	Of those really ill, how many we caught
F1 score	2 × precision × recall / (precision + recall)	0.727	Single balance of precision and recall

KEY IDEA **Choose the metric from the cost of mistakes**

Missing a sick patient (low recall) is worse than a false alarm, so recall matters most. Flagging an innocent customer as fraud (low precision) is costly in another way. Decide this **before** modelling, since it follows from the decision in Section 7.

11. Your first end-to-end experiment

The snippet below turns Illness into Y, splits the data, trains a logistic regression and compares it with a baseline that always predicts the most common class. Change the path to where the file lives in your copy of the repo.

```
PATH = "toy_dataset.csv"   # or the path to the file in 99_Practice_Notebook
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import accuracy_score, recall_score

df = pd.read_csv(PATH)
y = (df["Illness"] == "Yes").astype(int)      # dependent variable (Y)
X = df[["City", "Gender", "Age", "Income"]]   # independent variables (X)
print(y.value_counts(normalize=True).round(4).to_dict())

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0, stratify=y)

prep = ColumnTransformer([("cat", OneHotEncoder(), ["City", "Gender"]),
                          ("num", StandardScaler(), ["Age", "Income"])])
model = make_pipeline(prep, LogisticRegression(max_iter=1000)).fit(X_train, y_train)
baseline = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)

for name, m in [("baseline", baseline), ("model", model)]:
    pred = m.predict(X_test)
    print(name, "accuracy", round(accuracy_score(y_test, pred), 4),
          "recall", round(recall_score(y_test, pred), 4))
```

Output when run:

```
{0: 0.9191, 1: 0.0809}
baseline accuracy 0.9191 recall 0.0
model accuracy 0.9191 recall 0.0
```

The model equals the baseline: 91.9% accuracy and **0% recall**. It did not learn anything useful, because the X columns carry no signal about Illness (Section 7). This is the lesson to remember:

always compare with a baseline, and report recall or precision for rare classes.

12. A map of the algorithms you will meet

Algorithm	Used for	One-line idea
Simple and multiple linear regression	Regression	Fit a straight line or plane to the data
Gradient descent (SGD / batch)	Training method	Repeatedly adjust the parameters in the direction that reduces the loss; used by many models
Lasso (L1) and Ridge (L2)	Regression	Linear regression with a penalty on large weights to prevent overfitting; Lasso can zero out useless X's
Logistic regression	Classification	Despite the name, predicts the **probability of a class** with an S-shaped curve
K-nearest neighbours (KNN)	Both	Predict from the K most similar training rows
Decision tree	Both	A flowchart of yes/no questions
Random forest	Both	Many trees voting together
XGBoost and LightGBM	Both	Trees built one after another, each fixing the last one's mistakes
Naive Bayes	Classification	Uses probabilities, assuming X's are independent
Support vector machine / regressor	Both	Find the boundary or band with the widest margin
Neural network (ANN)	Both	Layers of simple units; the base of deep learning
Time-series regression	Regression	Forecast values ordered in time; split by date
K-means and hierarchical clustering	Clustering	Group rows by similarity
PCA	Dimensionality reduction	Replace many correlated columns with a few new ones

NOTE **Clarifying the class notes**

- **PCA** is dimensionality reduction (unsupervised), not a clustering algorithm itself; the notes rightly add that it is often used alongside clustering.
- **Multicollinearity** (redundant X's) and **overfitting** (memorising noise) are related but different problems; irrelevant or redundant X's are one common contributor to overfitting.
- **Binning students by score range** is grouping by fixed rules; a clustering algorithm finds the groups from the data without pre-set ranges.
- **'Discrete' versus 'continuous' Y:** a score from 0 to 100 has a limited range but is usually still modelled as a number (regression). It is a classification problem only when you predict a category such as Pass or Fail.
- **Classification** covers two classes (binary) **and** more than two (multi-class).

13. Common mistakes

- Choosing Y from what is easy to measure rather than what the decision needs.

- Modelling before looking at the data and before computing a baseline.
- Judging a model on its **training** data.
- Quoting accuracy alone on imbalanced data.
- Leaking Y into X, or scaling the whole dataset before splitting.
- Including IDs and redundant columns.
- Comparing scores of models that predict different Y's, or are tested on different rows.
- Believing a high score proves **cause**.
- Extrapolating far beyond the range of the training data.

SUMMARY

- ML finds rules from data and answers; the learned rules form the **model**, a function $\hat{Y} = f(X)$ with parameters learned by training.
- **Supervised** learning has a Y (regression: a number; classification: a category). **Unsupervised** has none (clustering, dimensionality reduction). Reinforcement learns by reward.
- **X** = inputs (many), **Y** = the one outcome to predict. Roles depend on the question.
- Choose Y from the **decision**, then test it: recorded, unknown at prediction time, varied, enough data, no leakage, predictable.
- Y decides the problem type, the metric, the ceiling on accuracy and the risk of leakage and imbalance.
- Always **split the data**, compare against a **baseline**, and pick metrics from the cost of errors.

Practice questions

Try each question before reading the answer underneath it.

Q1. What is the difference between an algorithm and a model?

Answer. The algorithm is the learning recipe (for example least squares). The model is the fitted result, the function with its learned parameters, that you use to predict.

Q2. A bank wants to know which loan applicants will not repay. Name Y, X examples and the problem type.

Answer. Y: defaulted within two years (yes/no). X: income, age, existing debt, employment length. It is classification.

Q3. You are asked to predict next month's sales for each store. What problem type is it and what is the baseline?

Answer. Regression, because Y is a number. A simple baseline is last month's sales or the average sales.

Q4. A churn model has 97% accuracy but only 2% of customers actually churn. Is it good?

Answer. Not necessarily: always predicting 'no churn' gives 98%. Check recall and precision for the churners.

Q5. Why is 'refund amount' a bad X for predicting whether an order will be returned?

Answer. The refund exists only after the return happens. It leaks the answer and will not be available at prediction time.

Q6. Is PCA supervised or unsupervised, and what does it do?

Answer. Unsupervised. It reduces many correlated columns to a few new ones.

Q7. You have only the columns Customer ID, Age, City and Spend and no known outcome. Which task fits?

Answer. With no Y, use clustering (for example segments by age and spend) or dimensionality reduction. Supervised learning needs a known Y.

Glossary

Term	Meaning
Feature	An input column (X)
Label / target	The column to predict (Y)
Training data	Rows used to fit the model
Test data	Rows held back to judge the model
Parameter	A number learned during training
Hyperparameter	A setting chosen before training
Loss function	A score of how wrong the model is
Baseline	The simplest prediction a real model must beat
Overfitting	Fitting noise; great on training, poor on new data
Underfitting	Too simple to capture the pattern
Data leakage	Training with information not available at prediction time
Inference	Using a trained model to predict new rows

Sources

- Google for Developers, Machine Learning Crash Course: ML terminology (examples, features, labels, models, training, inference). developers.google.com/machine-learning/crash-course/framing/ml-terminology
- scikit-learn documentation, 'Choosing the right estimator' (classification, regression, clustering, dimensionality reduction and the 50-sample guidance). scikit-learn.org/stable/machine_learning_map.html
- Machine Learning Mastery, 'Three subtle ways data leakage can ruin your models' (target leakage, train-test contamination, time leakage). machinelearningmastery.com
- T. Mitchell, *Machine Learning* (1997), definition of learning.
- This repo: 05_EDA class notes (variable identification, problem types, metrics) and 08_Statistics/03 regression notes.